

הפקולטה להנדסה
ע"ש איבי ואלדר פליישר
אוניברסיטת תל אביב



GPU-Accelerated Number Theoretic Transform (NTT)

Project Number: 3363

Final Project Report

Student: Kareem Bwakny

ID: 325126795

Student: Hakeem Najjar

ID: 206773111

Supervisor: Mr. Oren Ganon

Project Carried Out at:

Tel Aviv University — Iby and Aladar Fleischman Faculty of Engineering

Table of Contents

Abstract	2
1 Introduction	4
1.1 Goals	4
1.2 Motivation.....	4
1.3 Approach.....	4
1.4 Positioning relative to existing work	5
2 Theoretical background	6
2.1 The Number Theoretic Transform.....	6
2.2 Modular reduction.....	6
2.3 Why the NTT has no single bottleneck on a GPU.....	7
3 The RTX 2080 Ti architecture and how the NTT is mapped onto it	8
3.1 Hardware description.....	8
3.2 Mapping the transform onto the hardware	9
4 Analytical models and profiling	11
4.1 Kernel-launch model	11
4.2 Memory-traffic and roofline model.....	11
5 Implementation	13
6 Analysis of results.....	16
6.1 Throughput	16
6.2 The roofline: the kernels are memory-bound and saturated	17
6.3 Optimization gains	18
6.4 Where the speedup comes from	19
6.5 Bandwidth, the saturation boundary, and selector quality	19
7 Contribution relative to existing models.....	21
7.1 Head-to-head with the external GPU NTT reference	21
7.2 Contribution against the project's own baselines.....	22
7.3 Positioning against other published systems	22
8 Conclusions and further work	24
8.1 Suggestions for future work.....	25
9 Project Documentation	26
10 References	27

Abstract

The Number Theoretic Transform (NTT) is the modular-arithmetic analogue of the Fast Fourier Transform and the computational core of lattice-based (post-quantum) cryptography, homomorphic encryption, and zero-knowledge proof systems, where millions of transforms must be evaluated. The goal of this project was to accelerate the NTT on a GPU by studying its computational structure, identifying the hardware limit that binds performance in each problem regime, and mapping the transform onto the architecture of an NVIDIA RTX 2080 Ti (Turing, sm_75) accordingly. Building on an earlier Python→C++→PyTorch→Triton study, we designed a family of architecture-aware CUDA kernels and a compiled, table-driven runtime that dispatches the measured-best kernel for each problem size.

The work is organized around a single finding: the NTT does not have one bottleneck but a chain of them, and each problem regime is bound by a different hardware resource. Four kernel families each attack the limit that binds their regime — a warp-shuffle kernel that runs the first butterfly stages in registers with no barriers; a bank-conflict-free padded shared-memory kernel whose radix-8 butterflies cut synchronization barriers from 13 to 5; a multi-block 4-step kernel that spreads a single large transform across all 68 streaming multiprocessors; and a fused polynomial-multiply kernel that performs forward NTT, pointwise product, and inverse NTT in one launch. Pre-computed-quotient (Shoup) and Montgomery modular multiplication, block-size tuning, and vectorized memory access complete the design.

Every kernel is correctness-gated bit-exactly against an independent reference before it is timed. Architecture-aware tuning compounds to a 1.94× reduction in kernel execution time end to end; the saturated-regime kernels reach 86% of the measured DRAM roofline (up from 53% before tuning); peak throughput is 110.9 million transforms per second; and against an optimized C++ CPU implementation the speedup is up to 58.6×. Relative to a reproduced state-of-the-art open-source GPU NTT library, benchmarked on the same GPU with the same timing harness, the adaptive runtime is faster on 70 of 74 execution-verified (size, batch) configurations, up to 2.12×. The deliverable is a deployable, correctness-verified adaptive NTT runtime

together with a quantitative analysis of where each GPU optimization matters and why.

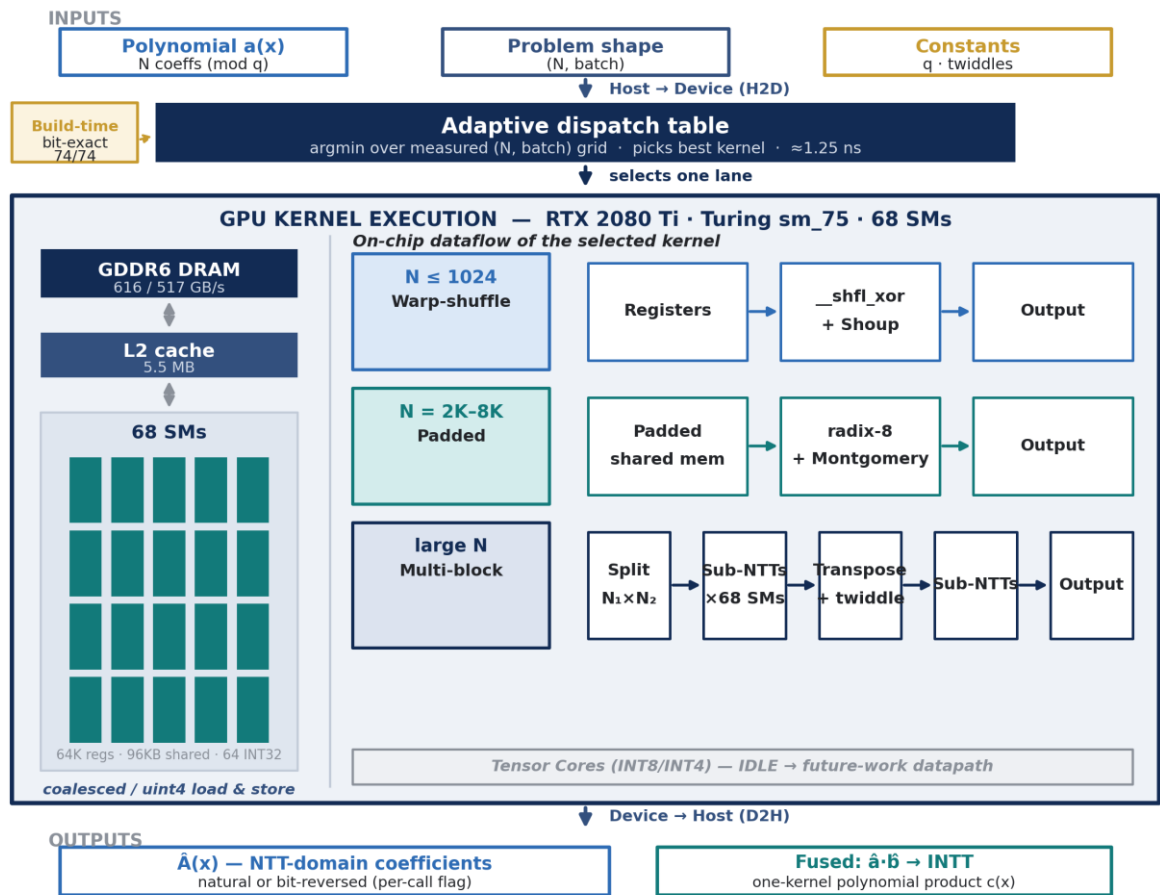


Figure 1: End-to-end block diagram of the adaptive CUDA NTT system. Inputs are copied to the device, a compiled dispatch table selects one kernel per (N, batch) request, and the GPU stage is shown opened to its memory hierarchy and the per-regime on-chip dataflow of each kernel.

1 Introduction

This chapter outlines the goals of the project, the motivation for accelerating the NTT on graphics hardware, the approach taken to the problem, and how the work is positioned relative to existing GPU NTT implementations.

1.1 Goals

The objective was to implement the forward cyclic NTT (modulus $q = 998244353$, primitive root $g = 3$, sizes $N = 256\text{--}16384$, batches $1\text{--}8192$) as specialized CUDA kernels for the RTX 2080 Ti; to determine, from measurement, which hardware resource limits performance in each regime; to optimize each regime with an architecture-specific technique; to fuse the transform into the polynomial-multiplication primitive that applications actually call; and to establish the contribution of the result both against the project's own CPU/earlier-GPU baselines and against a reproduced external CUDA NTT library on the same GPU, same modulus, and same timing harness — with correctness gated before every timing.

1.2 Motivation

Lattice-based cryptography (ML-KEM/Kyber, Dilithium), fully-homomorphic encryption (BFV, CKKS), and zero-knowledge proof systems all reduce their inner loop to polynomial multiplication over $\mathbb{Z}_q[x]/(x^n \pm 1)$, and the NTT turns that $O(n^2)$ product into an $O(n \log n)$ one. In these workloads thousands to millions of transforms are evaluated repeatedly, so the throughput and latency of a single NTT directly determine system performance. The transform is also structurally parallel — each of the $N/2 \cdot \log_2 N$ butterflies is independent within a stage — which makes the GPU the natural target, provided the computation is mapped onto the memory hierarchy and execution units correctly.

1.3 Approach

Rather than searching for a single “best” kernel, we treated the NTT as a set of distinct performance regimes. Controlled experiments (batch-scaling flatness tests, arithmetic-neutrality tests, block-count scaling, and barrier-

count ablations) were used to attribute each regime to its binding hardware constraint — launch/latency, shared-memory bank conflicts, synchronization-barrier cost, grid occupancy, twiddle-table footprint, or DRAM bandwidth — and a specialized kernel was then written to attack exactly that constraint. A measured dispatch table binds the kernels into one deployable system. Every performance claim traces to a committed raw measurement, and every kernel is validated bit-exactly before it is timed.

1.4 Positioning relative to existing work

The primary external reference is a state-of-the-art open-source CUDA NTT library (“Merge NTT”, Özcan & Savaş), rebuilt from source on the same node with the same compiler flags and modulus. It is the only external CUDA NTT reproduced experimentally at this transform scope on this hardware, and a dedicated chapter after the results (Chapter 7) quantifies our contribution relative to it. Other systems were considered and excluded for concrete, documented reasons: one ships a closed-source GPU NTT backend pinned to a different field; another targets sm_80+ hardware and cannot run on this sm_75 card; and tensor-core Kyber works use a different arithmetic model. Published numbers from those systems come from different GPUs and fields and are therefore not quoted as if measured here.

2 Theoretical background

2.1 The Number Theoretic Transform

The NTT is the discrete Fourier transform carried out in a finite field $\mathbb{Z}_q$ rather than over the complex numbers. For a prime $q = c \cdot 2^k + 1$ there exists a primitive N -th root of unity ω for every $N \mid 2^k$, and the forward transform of a length- N sequence x is $X[k] = \sum_n x[n] \cdot \omega^{nk} \bmod q$. Because the arithmetic is exact integer arithmetic, the NTT has none of the floating-point rounding error of an FFT, which is essential for cryptography. In this project $q = 998244353 = 119 \cdot 2^{23} + 1$, a 30-bit NTT-friendly prime with primitive root $g = 3$.

Like the FFT, the NTT is computed by a radix-2 Cooley–Tukey decomposition of $\log_2 N$ butterfly stages. Each butterfly reads a pair of coefficients, multiplies one by a twiddle factor ω^j (a modular multiplication), and forms a modular add and subtract. A length- N transform therefore performs $(N/2) \cdot \log_2 N$ butterflies. Two costs dominate: the modular multiplication on the butterfly critical path, and the strided data exchange between stages, each of which is bound by a different part of the GPU — a distinction that drives the whole design.

2.2 Modular reduction

The modular multiply on the butterfly critical path is the single most important arithmetic operation. A naive $(u64)a \cdot b \% q$ compiles to a long dependent sequence of integer operations. Three standard techniques shorten it. Barrett reduction replaces the division by a pre-computed reciprocal and two multiplies. The Shoup (Harvey) method exploits the fact that in an NTT one operand of every multiply is a fixed twiddle known at table-build time: it pre-computes a companion quotient $w' = \lfloor w \cdot 2^{32} / q \rfloor$ per twiddle so that at runtime the reduction is one high-multiply, one multiply-subtract, and a single conditional subtraction. Montgomery reduction keeps operands in a transformed residue domain and reduces with multiplies and shifts only, storing a single 32-bit constant per twiddle rather than the pair Shoup requires. As Chapter 6 shows, the better of Shoup and Montgomery depends

on the regime: Shoup shortens the dependency chain, while Montgomery's smaller twiddle table wins once the kernel becomes limited by twiddle-cache footprint at large N .

2.3 Why the NTT has no single bottleneck on a GPU

On a GPU the practical limits of the NTT are, in order of where they surface: (a) kernel launch and dependency latency at small sizes; (b) the integer modular-multiply dependency chain; (c) shared-memory bank conflicts in the strided cross-stage exchange; (d) the number and cost of block-level synchronization barriers once the data no longer fits inside a warp; (e) grid occupancy when one transform maps to a single block; (f) twiddle-table footprint in the on-chip caches; and (g) DRAM bandwidth once batches are large. Crucially, no single kernel is optimal across all (N, batch) because the binding constraint changes with the regime. This observation is the central design principle of the project: it motivates both a family of specialized kernels and, as the measurements in Chapter 6 make explicit, a chain in which removing one limit exposes the next.

3 The RTX 2080 Ti architecture and how the NTT is mapped onto it

This chapter describes the target GPU at the level of detail the kernel design depends on, and then shows how each part of the transform is placed onto a specific hardware resource. The hardware properties, and the design decision each one drives, are summarized in Table 1; Figure 2 opens up one streaming multiprocessor (SM) and labels each resource with the kernel that exploits it.

3.1 Hardware description

All development and benchmarking used a single NVIDIA GeForce RTX 2080 Ti (Turing TU102, compute capability 7.5), driver 550.120, CUDA 12.4, compiled with `nvcc -O3 -std=c++17 -gencode arch=compute_75,code=sm_75`. Turing exposes a separate, concurrent 64-wide INT32 datapath per SM, so the modular butterflies run as integer IMAD chains while the FP32 cores stay idle. The sm_75 generation has no `cp.async`, but it does carry second-generation tensor cores with INT8/INT4 support; the present design leaves that datapath idle, and Chapter 8 explains why and treats it as future work.

Table 1: RTX 2080 Ti hardware profile and its implications for the kernel design (measured via `cudaGetDeviceProperties` and a STREAM-style bandwidth probe).

Property	Value	Consequence for the NTT design
Architecture	Turing TU102, sm_75	Plain-SIMT INT32 design; no <code>cp.async</code> ; INT8/INT4 tensor cores left idle
SMs	68	One block per transform uses 1/68 of the GPU → multi-block kernel for large N
INT32 datapath	64-wide per SM, concurrent with FP32	Modular butterflies are integer IMAD chains on the INT32 units
Warp	32 lanes, up to 32 warps/SM	Early butterfly stages ($m=1..16$) fit in one warp → shuffle exchange, no barriers
Register file	64 K registers/SM	22–43 regs/thread → high occupancy; Shoup/Montgomery operand fits for free
Shared memory / L1	64 KB shared (96 KB unified) /SM	Single-block ceiling; 32 banks → padding rule; block size tunes L1 vs occupancy
L2 cache	5.5 MB	Twiddle tables (≤ 64 KB) stay resident → footprint, not refetch, is the limit
Memory	GDDR6, 352-bit → 616 GB/s peak	Measured (STREAM) roofline 517 GB/s; saturated cells reach 66–86% of it

Clocks	1545 MHz boost (300 MHz idle)	Timings taken at steady clocks after warm-up (WARM=5 / REP=101)
--------	-------------------------------	---

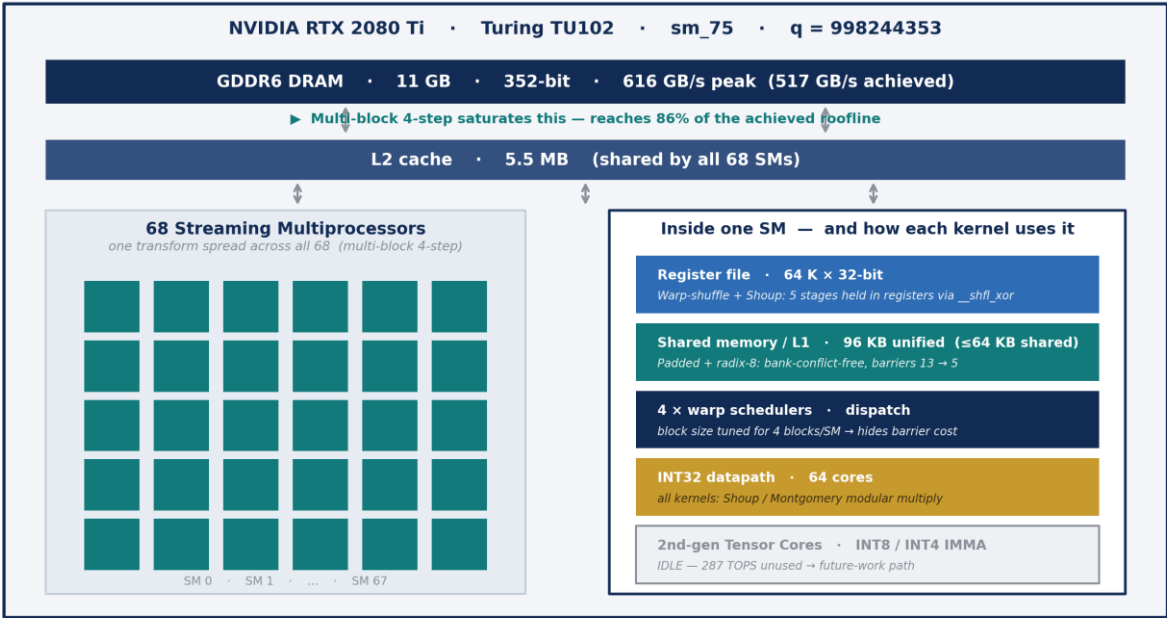


Figure 2: The RTX 2080 Ti memory hierarchy (GDDR6 → L2 → the 68 SMs) and, zoomed into a single SM, the on-chip resources. Each resource is annotated with the kernel that uses it: the register file for the warp-shuffle kernel, padded shared memory for the mid-N kernel, the warp schedulers for block-size tuning, and the INT32 datapath for modular arithmetic. The INT8/INT4 tensor cores are idle by design.

3.2 Mapping the transform onto the hardware

The design places each part of the transform on the resource that best fits it, and each placement is chosen to relieve the limit that binds its regime:

- Register file → small transforms. For $N \leq 1024$ the early butterfly stages pair elements within a single warp, so each lane exchanges its partner in registers with `__shfl_xor_sync`, with no shared-memory traffic and no barriers. The 64 K-register file also absorbs the extra Shoup operand for free at high occupancy.
- Shared memory and its banks → mid-size transforms. For $N = 2048\text{--}8192$ the transform is staged in shared memory; a padding word inserted per 32 elements removes bank conflicts,

and radix-8 butterflies perform three stages per barrier, cutting synchronization barriers from 13 to 5.

- Warp schedulers and occupancy → barrier cost. Block size is tuned so that several blocks reside on each SM at once (for example four 256-thread blocks instead of one 1024-thread block at $N = 2048$), so that a barrier stalls only a fraction of the SM rather than all of it.
- The INT32 datapath → modular arithmetic. Every butterfly multiply runs as an integer IMAD chain; Shoup or Montgomery reduction (whichever the regime prefers) keeps that chain short.
- All 68 SMs → large transforms. The multi-block 4-step kernel factors $N = N_1 \cdot N_2$ and spreads a single transform across every SM, so large- N low-batch work no longer idles 67 of 68 SMs.
- The L2 cache and DRAM → the saturated regime. Twiddle tables stay resident in the 5.5 MB L2, so at scale only coefficient traffic matters; vectorized (uint4) coalesced access drives the saturated kernels toward the measured DRAM roofline.

4 Analytical models and profiling

Two lightweight cost models were used to predict where time would be spent and to guide the kernel design. They play the role of the project's “simulation”: models validated against measurement.

4.1 Kernel-launch model

A butterfly-per-stage implementation issues one or more GPU operations per stage, so its launch count grows with $\log_2 N$. The earlier PyTorch-optimized implementation dispatched roughly seven framework operations per stage (71–99 launches for $N = 1024$ – 16384), whereas a fused kernel issues a single launch per transform — a $6.5\times$ reduction in dispatch overhead. This model predicted, correctly, that small transforms would be dominated by launch and dependency latency rather than arithmetic, and that fusing stages would be the first-order win.

4.2 Memory-traffic and roofline model

One forward NTT reads and writes 8 bytes per coefficient at steady state, with the twiddle table (≤ 64 KB) resident in the 5.5 MB L2 cache. Rather than compare against the 616 GB/s datasheet peak, a STREAM-style copy kernel was written to measure what this specific card actually sustains: 517 GB/s (534 GB/s device-to-device memcpy), i.e. 84% of the theoretical peak. This measured value is used as the roofline throughout the report; the theoretical peak is kept alongside, clearly labelled. The model separates two regimes: at small N the working set fits in L2 and the kernel is launch/latency-bound (low effective bandwidth by design); at large N and large batch the kernel is DRAM-bound. After tuning, the saturated kernels reach 66–86% of the measured roofline (Chapter 6).

One limitation is stated up front and revisited as future work. The benchmark cluster restricts hardware performance counters to administrators, so achieved occupancy, SM-active percentage, and stall reasons could not be captured with Nsight Compute during this work. Instead, theoretical occupancy is read from compiler-reported register and shared-memory usage, and the binding constraint of each regime is attributed by controlled

experiments — for example, wide (uint4) versus narrow global access to test whether a kernel is memory-access-bound, and barrier-count ablations to test whether it is synchronization-bound. Every metric in this report is labelled as measured, compiler-reported, analytically derived, or estimated.

5 Implementation

This chapter describes the family of CUDA kernels and the adaptive runtime that binds them together. The overall structure was shown in Figure 1: a request for an (N, batch) transform is routed by a compiled dispatch table to the kernel measured to be fastest for that regime, correctness is gated before timing, and the output layout is an explicit per-call contract. The kernel families are summarized in Table 2.

Table 2: The CUDA kernel families and the regime each one targets. All selected kernels are ptxas-verified with zero register spills.

Kernel family	Target regime	Key idea	Regs / spills
Warp-shuffle (+Shoup, radix-4)	$N \leq 1024$	Register-resident butterfly stages via <code>__shfl_xor_sync</code>	22 / 0
Padded shared memory (+radix-8)	$N = 2048\text{--}8192$	Bank-conflict-free SPAD(i)= $i + \lfloor i/32 \rfloor$; 3 stages/barrier	34–43 / 0
Multi-block 4-step (radix-4)	large N , low batch	$N = N_1 \cdot N_2$ split across all 68 SMs	45+37 / 0
Fused polynomial multiply	NTT · pointwise · INTT	One launch; DRAM traffic $9N \rightarrow 3N$ words	67.6 KB shared

Warp-shuffle kernel (small N). In a decimation-in-time NTT, stage st pairs elements at distance $m = 2^{\text{st}}$. For $m < 32$ both butterfly operands live in the same warp, so a lane exchanges its partner in registers with `__shfl_xor_sync` and the first five stages run with no shared-memory traffic, no bank conflicts, and no `__syncthreads()`. A radix-4 tail applies the same barrier-reduction idea to the shared-memory portion. Because small N is latency-bound, removing the shared-memory round-trip attacks exactly the critical path; the kernel reaches a $4.5\ \mu\text{s}$ latency floor.

Padded shared-memory kernel with radix-8 (mid N). For $N = 2048\text{--}8192$ the transform is staged in shared memory. The strided cross-stage exchange makes threads of a warp hit addresses congruent modulo 32, serializing them as bank conflicts; inserting one padding word per 32 elements (SPAD(i) = $i + \lfloor i/32 \rfloor$) shifts every group into distinct banks. On top of the conflict-free layout, radix-8 butterflies perform three NTT stages between each barrier, cutting the

barrier count at $N = 8192$ from 13 to 5. Block size is tuned so that several blocks reside per SM, so each remaining barrier stalls only part of the SM.

Multi-block 4-step kernel (large N , low batch). A single-block large- N kernel uses one SM's entire thread budget, so at low batch the other 67 SMs idle — the origin of the worst-case gap at large N . The 4-step decomposition factors $N = N_1 \cdot N_2$ (e.g. 128×128 at $N = 16384$) and runs N_1 sub-transforms, a fused twiddle multiply, then N_2 sub-transforms, turning one transform into many blocks that fill all 68 SMs. A radix-4 rewrite together with coalesced loads and a tuned block size makes this kernel $1.65\times$ faster than the prior multi-block design and, for the first time, competitive at $N = 16384$ across low batch.

Fused polynomial multiply. The primitive applications actually call is polynomial multiplication, not the forward transform alone. A fused kernel keeps both operands in shared memory and performs two forward transforms, the pointwise product, and the inverse transform in a single launch, reducing global-memory traffic from $9N$ to $3N$ words and four launches to one. It is gated for correctness against a CPU schoolbook cyclic convolution before any timing is reported.

Modular arithmetic — Shoup and Montgomery. Every butterfly multiplier is a twiddle known at table-build time. Shoup pre-computes a companion quotient and reduces in a short 32-bit sequence; Montgomery stores a single 32-bit constant per twiddle. Both are wired into the selector because the better choice is regime-dependent: Montgomery's smaller twiddle table wins above $N = 4096$, where the kernel becomes limited by twiddle-cache footprint rather than by the arithmetic itself.

Adaptive selector. The selector is a compiled, executed dispatch layer, not an offline argmin over a spreadsheet: a per-cell table keyed on $(\log_2 N - 8, \log_2 \text{batch})$ holds the measured-best backend, and a single indexed switch launches it. The measured lookup overhead is 1.25 ns, negligible against a $\geq 4.5 \mu\text{s}$ transform. The public API charges any output-layout mismatch (natural vs bit-reversed) to whichever backend needs the permutation, and exposes output order as an explicit per-call flag.

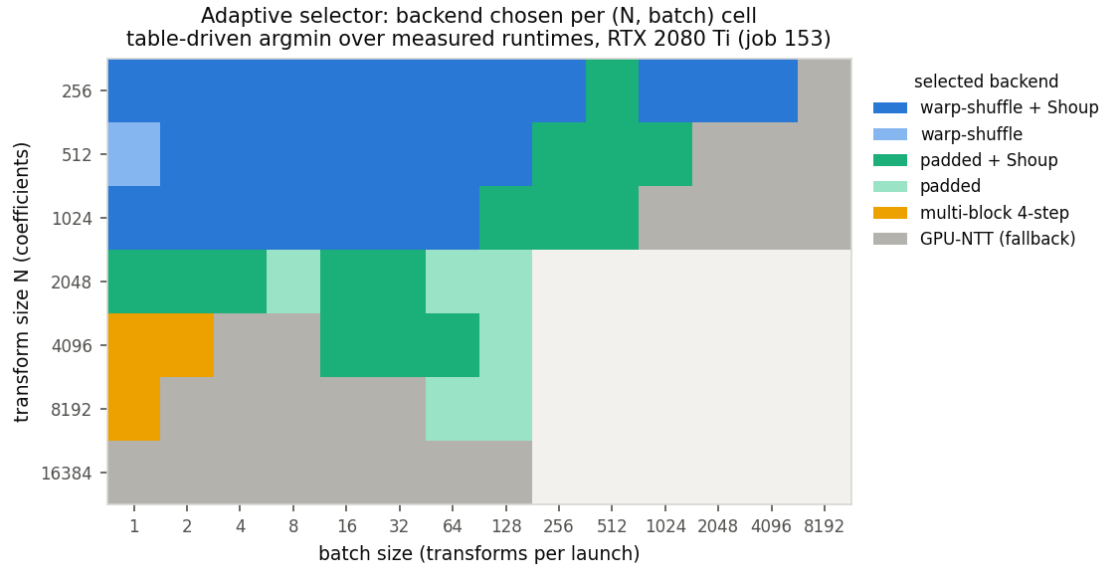


Figure 3: Backend selected by the adaptive runtime for each (N, batch) cell — the measured regime structure. Small and mid N are won by the warp-shuffle and padded kernels, large-N low batch by the multi-block kernel, and the few DRAM-saturated cells by a streaming fallback.

6 Analysis of results

All results below are compute-only (data GPU-resident, twiddles prebuilt), timed with CUDA events after warm-up (WARM = 5, REP = 101), and correctness-gated before timing on edge inputs (zeros, one-hot, all-(q-1), random) plus an inverse round trip, over $N = 256$ –16384. The transform convention was formally verified: our natural-order output is bit-exact with the direct DFT at $\omega = 3^{((q-1)/N)}$. The authoritative run (job 180) executed the selected backend for every one of the 74 (N , batch) cells with values and ordering both gated (74/74 pass). This chapter reports the system's own performance; the comparison with the external reference library is the subject of Chapter 7.

6.1 Throughput

Peak throughput is 110.9 million transforms per second, at $N = 256$, batch = 8192. Small transforms hit a 4.5 μ s latency floor set by launch and dependency latency, while at large batch each size reaches a DRAM-saturation knee, visible in Figure 4. Table 3 lists representative operating points spanning the regimes.

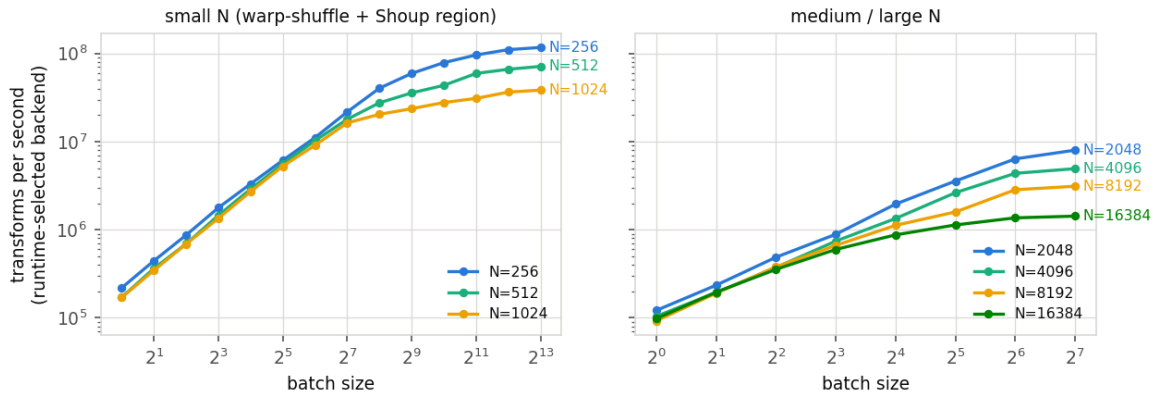


Figure 4: Throughput (transforms per second) versus batch size for each N , using the runtime-selected backend. The saturation knees where the kernel becomes DRAM-bound are clearly visible.

Table 3: Representative throughput of the runtime-selected backend (job 180).

Effective GB/s is a traffic estimate, not a hardware-utilization counter.

Workload	N	batch	Backend	μ s/NTT	NTT/s	est. GB/s
----------	-----	-------	---------	-------------	-------	-----------

small-N low batch	256	1	shuffle+Shou p	4.50	0.222 M	0.4
small-N pre-saturation	256	2048	shuffle+Shou p	0.010	98 M	201
small-N saturation	256	8192	shuffle+Shou p	0.009	110.9 M	446
medium-N	2048	32	padded+radi x-8	0.270	3.70 M	61
large-N low batch	16384	1	multi-block r4	8.10	123 K	16
large-N high batch	16384	128	multi-block r4	0.690	1.45 M	190

6.2 The roofline: the kernels are memory-bound and saturated

Figure 5 places each measured size on a roofline for the RTX 2080 Ti. The measured DRAM bandwidth ceiling is 517 GB/s and the INT32 compute ceiling is 6.4 TOPS, giving a ridge at an operational intensity of 12.3 operations per byte. Every size the project measures sits at an operational intensity of 4.5–7.9 ops/byte — to the left of the ridge — and reaches 66–86% of the bandwidth roof (86% at $N = 256$, falling to 66% at $N = 8192$). The kernels are therefore memory-bound and close to hardware saturation, which is the correct outcome for a transform of this arithmetic intensity and the reason no further arithmetic change helps in this regime.

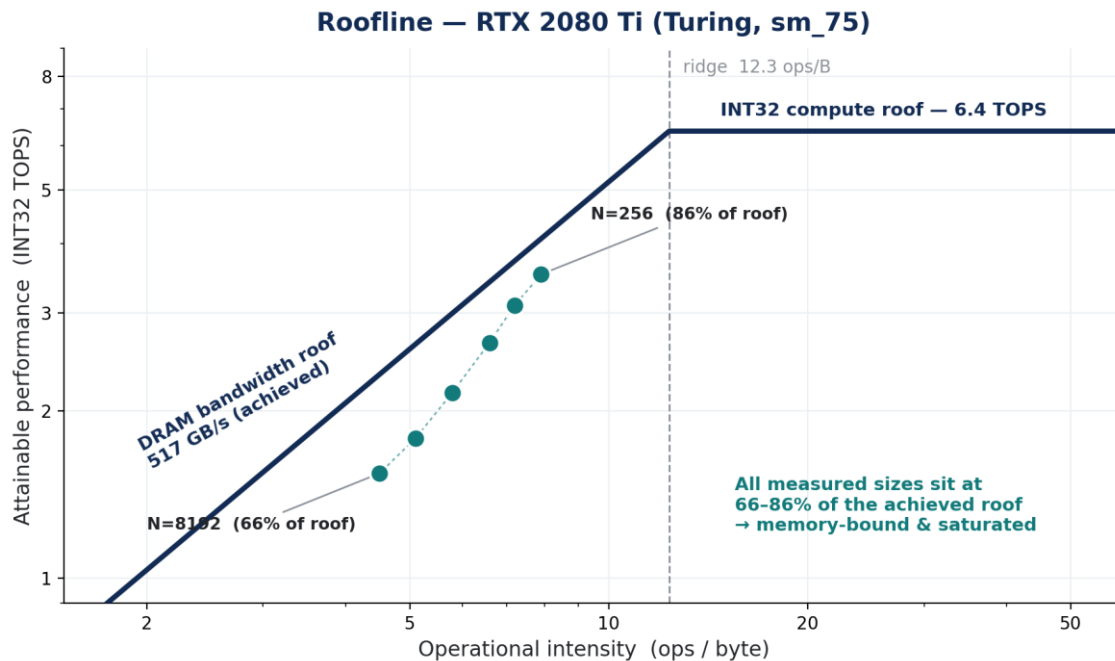


Figure 5: Roofline for the RTX 2080 Ti. Each point is one measured size; all sit at 66–86% of the measured 517 GB/s DRAM roof, confirming the kernels are memory-bound and saturated.

6.3 Optimization gains

Each optimization is deployed only in the regime where it was measured to help. Table 4 and Figure 6 isolate the contribution of each, measured round-robin (interleaved with its baseline to eliminate cross-run clock drift) and gated bit-exactly. Applied in sequence on a common set of cells, the techniques compound to a 1.94× reduction in kernel execution time.

Table 4: Contribution of each optimization, measured round-robin against the previous row on a common set of cells; all rows bit-exact.

Optimization	Measured gain	vs what	Category
Padded shared memory	1.235×	fused baseline	memory layout
Shoup butterfly	1.008×	prior row	arithmetic
Vectorized (uint2) I/O	1.012×	prior row	memory layout
Radix-4 butterfly	1.293×	prior row	barriers
Radix-8 butterfly	1.122×	prior row	barriers
Montgomery (large N)	1.016×	prior row	arithmetic
Block-size tuning	1.049×	prior row	occupancy
Combined (cumulative)	1.94×	fused baseline	all

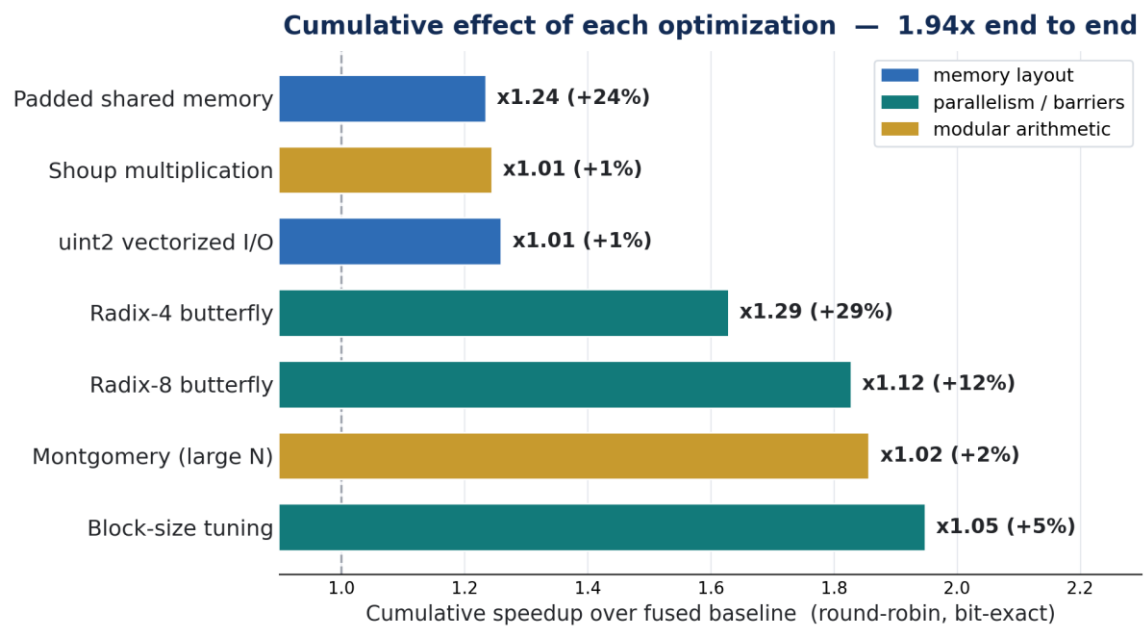


Figure 6: Cumulative effect of each optimization over the fused baseline, colored by the category of hardware limit it addresses. The two barrier-reduction steps (radix-4/8) are the largest levers.

6.4 Where the speedup comes from

Because the optimizations target different limits, it is informative to decompose the baseline kernel's execution time by the bottleneck each optimization removes. Figure 7 shows that of the baseline time, 27% is removed by reducing synchronization (radix-4/8 and block size), 20% by memory-layout changes (padding and vectorized access), and 2% by arithmetic, leaving the optimized kernel at 51% of the baseline — the 1.94×. This makes the central finding visible: synchronization cost, not arithmetic, was the dominant removable bottleneck in the mid-N regime.

Where the 1.94× comes from — baseline kernel time by the bottleneck each optimization removed

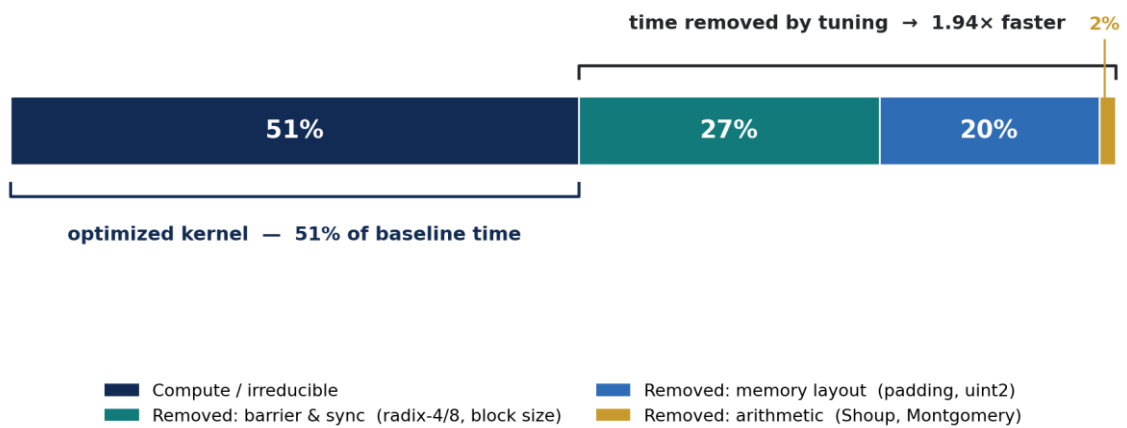


Figure 7: Baseline kernel execution time decomposed by the bottleneck each optimization removes. Barrier and synchronization cost is the largest removable component.

6.5 Bandwidth, the saturation boundary, and selector quality

Small-N low-batch cells run far below the DRAM roof by design (they are latency-bound), while saturated cells approach the measured roofline: the small-N saturated regime reaches 446 GB/s (86% of 517 GB/s), up from 272 GB/s (53%) before this tuning pass, and the mid-N saturated regime reaches 343 GB/s (66%), up from 204 GB/s (40%). Selector quality is quantified by regret (executed p50 ÷ separately measured best): the median is 1.000, i.e. the deployed runtime is at parity with the best available backend on the typical cell, with correctness 74/74. A methodological note: the original baseline used a shorter timing protocol (WARM = 3, REP = 9) that, with this card idling at

300 MHz, placed measurements inside the clock-boost ramp; all results here use the corrected WARM = 5 / REP = 101 protocol, and three fixed-order timing artifacts were identified and eliminated during the work.

7 Contribution relative to existing models

This chapter states the project's contribution against external and prior-art baselines. The comparison is deliberately separated from the self-contained results of Chapter 6 so that the system's own performance and its position relative to other work are each clear on their own terms.

7.1 Head-to-head with the external GPU NTT reference

The external reference (Merge NTT) was rebuilt from source on the same node, same modulus, and same timing harness, and both libraries were re-run in the same session under the corrected protocol so the numbers are directly comparable. Under the raw native-order contract — each library in its own output order, the fair setting for NTT→pointwise→INTT pipelines — the adaptive runtime is faster on 70 of 74 execution-verified cells, ties one, and is slower on three, with a maximum speedup of 2.12×. The three remaining losses are all at $N = 16384$ at high batch, the DRAM-saturated large- N corner; of these, only one cell differs by more than 3%, and there the same backend is selected on both sides, so the difference is run-to-run variance rather than a design deficit. Figure 8 places this against the state of the design before this optimization pass.

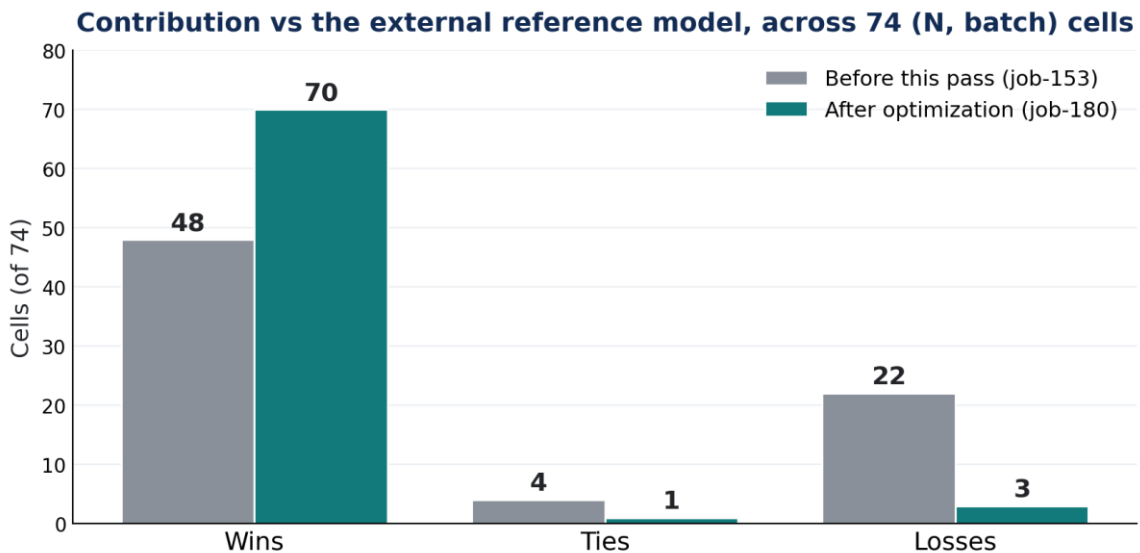


Figure 8: Win/tie/loss of the adaptive runtime versus the external reference across the 74-cell grid, before (48/4/22) and after (70/1/3) the architecture-optimization pass. $N = 16384$, previously never a win, now crosses parity (up to 1.27×).

The most significant change from prior state is at $N = 16384$: previously the project's worst regime, where the best result was $0.97\times$ (a loss), it now reaches $1.27\times$ after the multi-block radix-4 rewrite — the first time the large- N regime is won at all. Across the grid the runtime's specialized kernels are selected in 66 of 74 cells, and the median gain of the best-of-ours kernel over the prior baseline is $1.476\times$.

7.2 Contribution against the project's own baselines

Against the CPU and earlier-GPU implementations developed during the project, the gains are larger still. Table 5 summarizes the contribution against each baseline. The comparison against the optimized C++ CPU implementation is conservative: it is measured against the earlier kernel generation and is expected to increase once re-measured against the current kernels.

Table 5: Contribution of the final system against each baseline, with correctness gated bit-exactly in every case.

Baseline	Metric	Contribution
Optimized C++ CPU	throughput	up to $58.6\times$ faster
Earlier PyTorch/Triton GPU	kernel execution time	$1.94\times$ from architecture-aware tuning
External GPU NTT reference	74-cell win/tie/loss	70 / 1 / 3, up to $2.12\times$
Prior single-block large- N	multi-block redesign	$1.65\times$ (wins $N = 16384$)
Un-fused NTT·multiply·INTT	fused polynomial multiply	$1.3\text{--}1.9\times$, DRAM traffic $9N \rightarrow 3N$

7.3 Positioning against other published systems

Beyond the reproduced reference, several other GPU NTT systems exist but are not directly comparable on this hardware and field, and are therefore positioned qualitatively rather than quoted as if measured here. One widely used library ships a closed-source GPU NTT backend pinned to a different prime field; another targets sm_80+ hardware with features (such as asynchronous copies) that this sm_75 card lacks; and a line of tensor-core NTT work uses a digit-decomposed matrix-multiply arithmetic model on newer tensor hardware. The last of these is the most promising direction for extending the present design and is discussed as future work in Chapter 8.

8 Conclusions and further work

The project set out to accelerate the NTT on a GPU by understanding and then exploiting the hardware, and it succeeded. Its central finding is that the NTT is bound by a chain of distinct hardware limits, and that removing one exposes the next: reducing the number of synchronization barriers (radix-4/8) exposed the cost of each barrier (addressed by block-size tuning), which in turn exposed twiddle-table footprint (addressed by Montgomery at large N). Measured against the goals set at the outset:

- The transform was mapped onto the RTX 2080 Ti regime by regime: register-resident warp exchange for small N , conflict-free shared memory with radix-8 barrier reduction for mid N , and multi-block decomposition across all 68 SMs for large N at low batch.
- Architecture-aware tuning compounds to a $1.94\times$ reduction in kernel execution time, and the saturated kernels reach 66–86% of the measured DRAM roofline, up from 40–53% before the pass.
- The fused polynomial-multiply kernel brings the optimization to the primitive applications actually call, cutting global traffic from $9N$ to $3N$ words and four launches to one.
- Against the external GPU NTT reference the adaptive runtime wins 70 of 74 execution-verified configurations, up to $2.12\times$, and for the first time wins the large- N ($N = 16384$) regime; against an optimized C++ CPU baseline the speedup is up to $58.6\times$.
- Correctness was never traded for speed: every kernel passes edge, random, and round-trip gates bit-exactly before it is timed, and the output convention was formally verified.

The main trade-offs are understood and, in context, minor relative to the gains. The three remaining losses are same-backend run-to-run variance in the DRAM-saturated corner, not a design deficit. Shoup doubles the twiddle table and the multi-block kernel doubles global traffic, so each is deployed only in its measured win region; the selector makes those choices automatically. The one honest ceiling is profiling: because the cluster restricts

hardware counters to administrators, achieved occupancy and stall breakdowns were attributed by controlled experiment rather than directly measured — the first item of future work below would close exactly that gap.

8.1 Suggestions for future work

- Obtain administrator-enabled hardware profiling (Nsight Compute) to replace experimentally-attributed occupancy and stall reasons with directly measured evidence.
- Exploit the idle second-generation tensor cores: express the small sub-transforms of the multi-block kernel as digit-decomposed INT8 matrix multiplies, following current tensor-core NTT literature, and evaluate against the plain-INT32 path.
- A persistent small-N megakernel that keeps many transforms resident to push past the current 4.5 μ s launch floor, and self-calibration of the dispatch table on first run so the runtime adapts to a new GPU automatically.
- Applying the same principles at protocol scale: a preliminary Kyber ($q = 3329$) case study already shows a 12.3 $\times$ arithmetic-core improvement from transform reuse, kernel fusion, and NTT-domain accumulation, pointing to a full ML-KEM pipeline as the natural next step.

9 Project Documentation

All project materials — source code for every kernel, the adaptive runtime, the build and correctness harnesses, the raw benchmark CSVs, and the plotting scripts — are kept under version control. This report references them but does not reproduce code listings or raw data pages. The optimization pass is preserved as a tagged baseline plus a dedicated branch, so every reported change is reproducible against the exact pre-optimization state.

- Repository: [insert GitHub/GitLab link].
- Source (tools/): warp-shuffle, Shoup/Montgomery, padded shared-memory with radix-4/8, multi-block 4-step, and fused polynomial-multiply CUDA kernels; the adaptive runtime dispatch layer; and the correctness/convention harness.
- External baseline (external/): the reproduced Merge NTT library, pinned commit and build log, modulus fixed to 998244353.
- Results (results/raw/): committed benchmark CSVs for the 74-cell run (job 180), the roofline probe, the ablation, and representative throughput; the pre-optimization baseline is preserved separately and untouched.
- Figures (results/plots/): the plots reproduced in this report, regenerated from the raw CSVs by the plotting scripts.

10 References

- [1] J. W. Cooley and J. W. Tukey, “An algorithm for the machine calculation of complex Fourier series,” *Mathematics of Computation*, vol. 19, no. 90, pp. 297–301, 1965.
- [2] V. Shoup, “A Computational Introduction to Number Theory and Algebra,” Cambridge University Press, 2nd ed., 2009 (pre-computed-quotient modular multiplication).
- [3] D. Harvey, “Faster arithmetic for number-theoretic transforms,” *Journal of Symbolic Computation*, vol. 60, pp. 113–119, 2014.
- [4] P. L. Montgomery, “Modular multiplication without trial division,” *Mathematics of Computation*, vol. 44, no. 170, pp. 519–521, 1985.
- [5] P. Barrett, “Implementing the Rivest–Shamir–Adleman public key encryption algorithm on a standard digital signal processor,” *CRYPTO 1986*, LNCS 263, pp. 311–323.
- [6] A. Özcan and E. Savaş, “GPU-NTT: An Efficient GPU Implementation of the Number Theoretic Transform,” (Merge NTT), open-source: github.com/Alisah-Ozcan/GPU-NTT.
- [7] R. Longa and M. Naehrig, “Speeding up the Number Theoretic Transform for Faster Ideal Lattice-Based Cryptography,” *CANS 2016*, LNCS 10052, pp. 124–139.
- [8] R. Avanzi et al., “CRYSTALS-Kyber Algorithm Specifications (ML-KEM / FIPS 203),” NIST Post-Quantum Cryptography Standardization, 2024.
- [9] K. Sugizaki and D. Takahashi, “Improved Implementation of the Number Theoretic Transform on NVIDIA GPUs with Tensor Cores,” *HPCAsia 2026*.
- [10] NVIDIA, “CUDA C++ Programming Guide” and “Turing Tuning Guide,” NVIDIA Corporation, 2024.